

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Sasikumar Megha	Reg. No.:	192471038
Section / Batch:	3 rd CSK BS	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

95

Code of Conduct

I, Sasikumar Megha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%	✓			
Comparison & Differentiation	25%	✓			
Critical Evaluation	25%	✓			
Synthesis & Insight Generation	20%		✓		
Clarity	10%	✓			
Total Marks (100):		95			

Faculty In-charge	Course Coordinator	Head of Department
Name: <u>Riazuldeen</u>	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: <u>31/8/21</u>	Date: _____	Date: _____

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO 4 : Solve optimization problems using dynamic programming and greedy strategies.

Levels of Rubrics

(BL-4, BL-5)

Assessment Tool Weightage: 10%

QUESTIONS: 10 Total Marks: 50

Annexure A

Comparative Analysis

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Comparative Analysis

Q90. Optimal BST vs Binary Search

Introduction

Both Optimal Binary Search Tree (Optimal BST) and standard Binary Search are used to efficiently search sorted keys. However, they have different goals.

Comparison

Feature	Optimal BST	Standard Binary Search
Main goal	Minimize expected search cost	Find a key quickly
Structure	Special BST based on access probabilities	Sorted array
Input	Keys + search frequencies	Sorted keys
Strategy	Dynamic Programming	Divide and Conquer
Search complexity	Average can be minimized	$O(\log n)$
Construction	$O(n^3)$ basic DP	No construction required
Best use	Unequal/frequent searches	Uniform or simple searches

Cost Optimization

Standard Binary Search generally divides the sorted array into two equal parts.

Optimal BST considers **how frequently each key is searched**. Frequently accessed keys are placed closer to the root, reducing the average search cost.

For example, if key 50 is searched much more frequently than other keys, an Optimal BST may place 50 near the root.

Complexity

- Binary Search: **$O(\log n)$** per search.
- Optimal BST construction: **$O(n^3)$** using the basic DP method.
- Optimal BST search: **$O(h)$** , where h is tree height.

Scalability

Binary Search scales very well for large sorted arrays because it requires only logarithmic search time. Optimal BST becomes more useful when search frequencies are highly unequal, although constructing the tree is computationally expensive.

Conclusion

Binary Search is preferable for simple and frequently changing datasets. **Optimal BST** is better when access frequencies are known and reducing the **average search cost** is more important.

Q91. Huffman Coding vs Container Loading

Introduction

Huffman Coding and Container Loading are optimization problems that can use **greedy algorithms**. In both cases, decisions are made step-by-step based on the best current choice. **Comparison**

Feature	Huffman Coding	Container Loading
Objective	Minimize encoded data length	Maximize loaded value/space usage
Greedy choice	Combine two lowest frequencies	Select suitable items
Data structure	Min-heap	Usually sorting/list
Approach	Greedy	Greedy heuristic in general
Correctness	Produces optimal prefix code	Greedy may not always be optimal
Typical complexity	$O(n \log n)$	Depends on strategy

Greedy Strategy

Huffman Coding:

1. Find two symbols with the smallest frequencies.
2. Combine them.
3. Insert the combined node back.
4. Repeat until one tree remains.

This produces an optimal prefix code.

Container Loading:

Items may be selected according to criteria such as:

- highest value,
- smallest weight, • value/weight ratio,
- available capacity.

The choice depends on the specific loading problem.

Correctness

Huffman Coding has a proven greedy-choice property, so its greedy strategy produces an **optimal prefix code**.

Container loading is generally related to packing/knapsack-type problems. A simple greedy strategy **does not always guarantee an optimal solution**.

Complexity

Huffman Coding using a min-heap:

$O(n \log n)$

Container loading complexity depends on the selected algorithm and sorting method, commonly around $O(n \log n)$ for a sorting-based heuristic.

Scalability

Huffman Coding scales efficiently to large numbers of symbols. Container loading becomes harder as the number of containers and items increases because many combinations may need consideration.

Conclusion

Huffman Coding demonstrates a **greedy method with guaranteed optimality**, while Container Loading demonstrates that greedy techniques can be efficient but **may sacrifice optimality**.

Q92. Knapsack vs Job Sequencing

Introduction

Knapsack and Job Sequencing are both optimization problems, but they commonly demonstrate different algorithmic approaches.

Comparison

Feature	Knapsack	Job Sequencing
Typical approach	Dynamic Programming	Greedy
Objective	Maximize value within capacity	Maximize profit before deadlines
Main constraint	Weight/capacity	Deadlines/time slots
Decision	Include/exclude items	Select and schedule jobs
Optimality	DP guarantees optimal solution	Greedy can guarantee optimal solution for standard job sequencing
Complexity	$O(nW)$ for 0/1 Knapsack	$O(n^2)$ basic approach

Knapsack using DP

For **0/1 Knapsack**, each item can either be selected or rejected.

The recurrence is:

$DP[i][w] = \max(DP[i-1][w], \text{value}[i] + DP[i-1][w - \text{weight}[i]])$ when the item fits.

This systematically evaluates possible choices.

Job Sequencing using Greedy

The standard strategy is:

1. Sort jobs by decreasing profit.
2. Select the highest-profit job.
3. Place it in the latest available slot before its deadline.
4. Continue until all possible jobs are scheduled.

Correctness Difference

Dynamic Programming explores overlapping subproblems and guarantees the optimal result for 0/1 Knapsack.

The greedy strategy for **standard unit-time Job Sequencing with deadlines and profits** also gives an optimal solution because selecting high-profit jobs and placing them as late as possible preserves earlier slots. **Scalability**

Knapsack DP depends on capacity W , so very large capacities can make it expensive.

Job Sequencing is generally more scalable because sorting and scheduling can be performed efficiently.

Conclusion

Use **Dynamic Programming** when decisions depend on combinations and overlapping subproblems. Use **Greedy** when the problem has a valid greedy-choice property.

Q93. Dice Throw vs Word Wrap

Introduction

Dice Throw and Word Wrap are Dynamic Programming problems, but their DP tables represent different types of states.

Comparison

Feature	Dice Throw	Word Wrap
Goal	Count ways to obtain a target sum	Minimize line-breaking cost

DP type	Counting	Optimization
State	Number of dice and sum	Words considered and line position
Feature	Dice Throw	Word Wrap
Recurrence	Sum of previous states	Minimum cost among valid breaks
Output	Number of combinations/ways	Minimum formatting cost

Dice Throw DP

Let:

$dp[i][j]$ = number of ways to obtain sum j using i dice.

Recurrence:

$dp[i][j] = \sum dp[i-1][j-k]$ where k
represents a possible face value.

The table grows according to **number of dice** $\times$ **possible sums**.

Word Wrap DP

Let:

$dp[i]$ = minimum cost for arranging the first i words.

For each possible previous break position j , calculate the unused space and its cost.

A simplified recurrence is:

$dp[i] = \min(dp[j] + \text{cost}(j+1, i))$

Structure Difference

Dice Throw:

Previous dice $\rightarrow$ **possible face** $\rightarrow$ **new sum**

Word Wrap:

Previous word break $\rightarrow$ **line arrangement** $\rightarrow$ **minimum cost**

Therefore, Dice Throw mainly performs **counting**, while Word Wrap performs **optimization**.

Complexity

For n dice, target sum S , and F faces:

- Dice Throw: approximately **$O(nSF)$**
- Word Wrap: typically **$O(n^2)$**

Scalability

Dice Throw becomes expensive when the number of dice and target sum become very large. Word Wrap becomes expensive when the number of words increases, although optimized implementations can improve practical performance.

Conclusion

The key lesson is that DP tables depend on the **state definition**. Dice Throw stores the number of ways, whereas Word Wrap stores the minimum cost.

Q94. Bellman-Ford vs Warshall

Introduction

Bellman-Ford and Warshall are graph algorithms, but they solve different problems. **Comparison**

Feature	Bellman-Ford	Warshall
Main purpose	Single-source shortest path	Transitive closure/reachability
Output	Distance from source	Whether paths exist between vertices
Graph type	Directed/weighted	Usually directed graph
Negative edges	Supported	Reachability does not depend on weights
Negative cycles	Can detect	Not its primary purpose
Complexity	$O(VE)$	$O(V^3)$

Bellman-Ford

Bellman-Ford calculates the shortest distance from **one source vertex** to all other vertices.

It repeatedly relaxes every edge: $d[v]$

$$= \min(d[v], d[u] + w(u,v))$$

It can handle **negative edge weights**.

It can also detect a reachable negative-weight cycle.

Warshall

Warshall determines whether a path exists between every pair of vertices.

Its recurrence is:

$$R[i][j] = R[i][j] \text{ OR } (R[i][k] \text{ AND } R[k][j])$$

It produces a **reachability matrix**.

Output Difference Example:

Bellman-Ford: $A \rightarrow B = 5$ means the shortest distance from A to B is 5.

Warshall: $R[A][B] = 1$ means

B is reachable from A.

Scalability

Bellman-Ford is useful for sparse graphs but becomes expensive with many edges.

Warshall requires $O(V^3)$ time and is generally suitable for smaller or moderately sized graphs where all-pairs reachability is required.

Conclusion

Use **Bellman-Ford for single-source shortest paths**, especially when negative edges exist. Use **Warshall for all-pairs reachability**.

Q95. Prim vs Dijkstra

Introduction

Prim's and Dijkstra's algorithms both use a greedy strategy and repeatedly select a vertex with the smallest current value. However, their objectives are completely different.

Comparison

Feature	Prim's Algorithm	Dijkstra's Algorithm
Objective	Minimum Spanning Tree	Shortest paths
Starting point	Any vertex	Source vertex
Greedy value	Minimum edge connecting tree	Minimum distance from source
Output	MST	Shortest-path distances/tree
Negative edges	Edge weights generally can be negative	Not valid with negative edges
Complexity with binary heap	$O(E \log V)$	$O(E \log V)$

Prim's Greedy Selection

Prim selects the **minimum-weight edge** that connects a vertex outside the current tree to a vertex already inside the tree.

Example:

A --2-- B --3-- C

Prim chooses the cheapest connecting edge while building one connected tree.

Dijkstra's Greedy Selection

Dijkstra selects the unvisited vertex with the **smallest known distance from the source**.

For example, if:

$A \rightarrow B = 4$ and

$A \rightarrow C = 2$

Dijkstra first processes C because its current distance is smaller.

Objective Difference

Prim:

"What is the cheapest way to connect all vertices?"

Dijkstra:

"What is the shortest path from the source to each vertex?"

Correctness

Prim is correct for constructing a Minimum Spanning Tree when edge weights satisfy the usual MST assumptions.

Dijkstra is correct when edge weights are **non-negative**.

Scalability

Both algorithms can be implemented efficiently using priority queues.

- Prim: $O(E \log V)$
- Dijkstra: $O(E \log V)$

For dense graphs, array/matrix implementations may have $O(V^2)$ complexity.

Conclusion

Although both use greedy node/edge selection, their purposes differ:

Prim $\rightarrow$ Minimum Spanning Tree

Dijkstra $\rightarrow$ Shortest Path

The important insight is that **the same greedy idea—selecting the smallest current value—can solve different problems depending on what that value represents.**